Implementing A Mamba-Based Imitation Learning Policy For Pick-And-Place

1st Alex Hovakimyan
Department of Computer Science
San Jose State University
San Jose, USA
alex.hovakimyan@sjsu.edu

2nd Andranik Avagyan
Department of Computer Science
San Jose State University
San Jose, USA
andranik.avagyan@sjsu.edu

***Abstract*—We present an implementation of the Mamba Temporal Imitation Learning (MTIL) policy on the LeRobot platform, evaluated on a real world ball pick-and-place task using the SO-101 leader and follower arms. Our implementation reduces the hardware footprint required to train MTIL from a a 4090 with 128 GB of RAM down to a single RTX 5070 Ti with 32 GB, while still remaining faithful to the architectural choices of the original paper. On a 100 episode dataset collected with 2 cameras at 224x224(as opposed to the 4 640x480 cameras used in the original paper), we observe that MTIL underperforms ACT in this low data regime and exhibits severe overfitting after roughly 7,500 of 25,000 training steps. We further show that allowing the policy to retry after a failed grasp lifts success rates for both architectures, though our 10 trial sample size limits the precision of these estimates.**

I. Introduction

Transformer based visuo-motor policies such as ACT [2] have become a dominant choice for short imitation learning tasks, but their reliance on attention over a fixed context window makes them awkward for tasks whose state is not well captured by a Markov decision process(MDP). Manipulation tasks, especially those involving step-by-step procedures (slicing a cucumber, folding a shirt, plating a dish), demand memory that extends well beyond the few-frame chunking that attention can comfortably ingest at every timestep.

Mamba [3] and its successor Mamba2 [4] address this by replacing self-attention with a selective state space model whose hidden state compresses the entire history at constant, linear cost. Mamba has been deployed with success in language modeling, audio, genomics, and forecasting, and most recently in robotics through the MTIL policy [1], which couples Mamba2 with a frozen DINOv2 [5] encoder to produce action chunks for bimanual manipulation. The appeal for long-horizon manipulation is direct: a recurrent state that costs nothing to extend gives the policy a path to remember context that an attention window cannot hold.

This paper contributes an open-source MTIL implementation integrated with the LeRobot platform [6], a set of engineering changes that bring the training cost into the range of a single mid consumer GPU, and a small empirical study of MTIL versus ACT on a real world ball pick-and-place task collected with the SO-101 arm.

II. Choosing A Policy

We chose MTIL over alternatives such as Diffusion Policy or larger VLA models for three reasons. First, the architecture is light enough to bring within reach of consumer hardware: with the parameter and memory adjustments described in Section III, training fits on a single RTX 5070 Ti rather than the RTX 4090 used in the original work, which in turn is far less compute than of that required by VLAs like pi0.5. Second, the original paper reports that MTIL outperforms ACT on bimanual cube transfer and insertion, which makes the policy a credible candidate for harder, long-horizon tasks once the integration work is done. Third, no Mamba-based policy currently ships with LeRobot, so a working implementation immediately benefits any user with a LeRobot compatible dataset, robot, or cloud checkpoint, and provides a baseline for Mamba research in the ecosystem.

III. Implementation Tradeoffs

Our implementation is faithful to the load bearing pieces of the original design: DINOv2-Large layer-(-4) spatial features, d_{model}=2048, d_{state}=512, headdim=128, four Mamba2 layers, a 16-head cross camera self attention block followed by an 8-head cross modal block, AdamW with learning rate 2×10^{-4} and weight decay 5×10^{-4}, a frozen visual encoder, and an action chunk of 16. The deviations cluster in four places, each motivated by either compute budget or ergonomics within LeRobot.

A. Image Resolution

We train at 224×224 rather than the paper's 640×480. With DINOv2's patch-14 backbone this yields a 16×16=256-token spatial grid per camera, against roughly $1{,}564$ tokens per camera in the original setting. Because the cross camera self attention concatenates these tokens before mixing, the original policy reasons over roughly six times as many spatial slots. For pick-and-place of moderately sized objects, this matters less than one might fear: a patch covers about 20 pixels of source image, which is sufficient to localize the target but begins to lose edges, contact points, and small parts. The `dinov2_image_size` field exposes 308 and 392 as escape hatches for finer-grained tasks, with quadratic cost in DINOv2 throughput.

B. State Noise

The paper injects state noise as $0.02 \times \sigma_{\text{joint}}$ on raw radians. We instead inject 0.02 flat in normalized units, since LeRobot's MEAN_STD normalizer already divides by the feature standard deviation. The two are mathematically equivalent only when the dataset standard deviation matches the analytical joint standard deviation, which is approximately but not exactly true. Because the original paper identifies state noise as the policy's primary regularizer, the practical consequence is that joints with little relevant motion (for example, base rotation in a stationary setup) receive proportionally more perturbation under our scheme than under the paper's. The effect is subtle but worth flagging for tasks where some degrees of freedom are nearly static.

C. Parallel Episode Batching

The paper trains with one full episode per scan; we default to two episodes side by side along the batch dimension. This roughly halves gradient variance while leaving the learning rate unchanged from the paper's $B{=}1$ configuration. Linear scaling rules would suggest $4{\times}10^{-4}$ at $B{=}2$, which we do not apply. In a 100 episode data regime, this is plausibly beneficial rather than harmful: smoother gradients reduce the stochastic noise that would otherwise let the model memorize individual episodes, at the cost of slightly slower convergence.

D. Image Caching

We decode the entire dataset beforehand into approximately 16 GB of `uint8` tensors held in RAM, replacing the per step decode path. The pixels are identical, so the policy sees no behavioral change, but caching forces `image_transforms.enable=False` because cached frames are deterministic across epochs. The original paper also trains without image augmentation, so we remain paper faithful. The tradeoff is that augmentation is no longer a free regularization knob, since enabling it requires reverting to live decoding, which our profiling identified as the training bottleneck.

IV. Setup And Data Collection

The hardware setup uses the SO-101 leader and follower pair, with the operator teleoperating the leader arm to record trajectories on the follower. Two 1080p USB cameras provide visual input: an overhead camera looking straight down for a strategic view of the workspace, and a shoulder mounted camera on the arm pointing forward for an egocentric view of the gripper and target. Both streams are cropped to 480p for collection and further resized to $224{\times}224$ for training. This was done to balance visual fidelity against the memory cost of full resolution caching. The task is picking a ball from a randomized starting position on the table and placing it into a fixed bin. We collected 100 demonstration episodes, all initiated from the same neutral resting pose, with no trajectories starting from failure included.

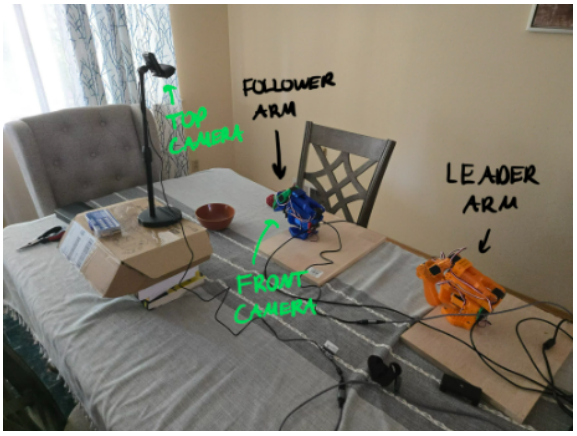
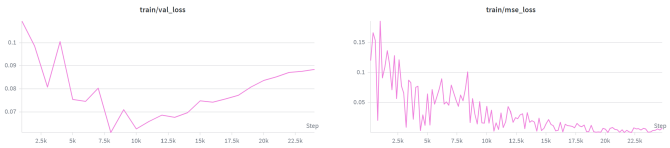


Fig. 1: Hardware setup. SO-101 follower arm with overhead and shoulder-mounted cameras over the pick-and-place workspace.



(a) Validation loss. (b) Training MSE loss.

Fig. 2: Loss curves over 25,000 training steps. Validation reaches its minimum at step 7,500 while training MSE continues to descend, indicating overfitting.

V. Training And Wandb Analysis

We trained for 25,000 steps and logged validation MSE on a held out split throughout. The validation curve reaches its minimum at approximately step 7,500 and rises monotonically afterward, while training MSE continues to fall toward values one to two orders of magnitude smaller. This is textbook overfitting.

We attribute the gap primarily to the combination of dataset size and architectural capacity. Mamba2 is a pretty high bandwidth sequence model: the selective state can encode a great deal of episode-specific information, and with only 100 episodes there is not enough signal to prevent that capacity from being spent on memorization. This is especially true since we are using only 2 cameras as opposed to 4 as in the original paper. Reducing the DINOv2 input resolution mitigated the problem somewhat by lowering effective visual dimensionality, but did not eliminate it. Because the visual encoder is frozen, conventional remedies that act on the encoder (heavier augmentation, encoder dropout) are unavailable, and we noted earlier that aggressive caching closes off the augmentation route.

For evaluation we selected the step 7,500 checkpoint rather than the final one, on the standard rationale that minimum validation loss is the best estimator of generalization to unseen rollouts.

TABLE I: Pick-and-place success rate, 10 trials per policy.

Policy	Success (%)
ACT	10.0
ACT + Fail Restarts	60.0
MTIL	0.0
MTIL + Fail Restarts	20.0

TABLE II: Reference results from the original MTIL paper [1].

Policy	History	Cube (%)	Insertion (%)
ACT	1 (Markov)	90.0 ± 2.0	50.0 ± 3.5
MTIL (10 step)	10	92.0 ± 1.5	56.0 ± 2.5
MTIL (full)	400	100.0 ± 0.0	84.0 ± 2.1

VI. Results

We evaluated each policy over 10 independent rollouts. A rollout was scored as a success if the ball was placed in the bin on the first attempt. We report two protocols: a protocol which allows the policy to retry after failing once, which matches the standard evaluation in the literature, and a fail restart protocol in which the arm is returned to its neutral pose after a failed grasp.

The most striking pattern is the gap between the single attempt and fail restart columns of Table I. Returning the policy to its neutral position after a missed grasp lifts ACT from 10% to 60% and MTIL from 0% to 20%, which we attribute to the absence of recovery trajectories in the training set. Every demonstration begins from the same neutral pose, so when a rollout enters a configuration that resembles a mid task failure rather than a nominal start, the policy is effectively out of distribution and has no learned behavior to fall back on. Resetting the arm pulls the input distribution back to one that the policy has seen, and a meaningful fraction of subsequent attempts succeed.

The absolute numbers also fall well short of the paper's results in Table II, and we believe two factors explain most of the gap. The first is the implementation tradeoffs of Section III, particularly the resolution reduction, which compresses the spatial grid by roughly $6\times$ and is most likely to bite on small object localization. The second, and more important one, is camera count: we use 2 cameras to the paper's 4, halving the visual frames available per timestep and effectively halving the regularization that comes from cross view consistency. Combined with a 100 episode dataset, this is enough to produce the overfitting pattern of Section V, and the relative ordering of MTIL below ACT in our results is consistent with MTIL's higher capacity being a liability rather than an asset with such little data.

VII. Conclusion And Future Work

The principal contribution of this work is engineering rather than algorithmic: a faithful, open source MTIL implementation that integrates with LeRobot and trains on a single RTX 5070 Ti with 32 GB of system RAM, against the original's RTX 4090 and 128 GB. This lowers the barrier to entry for researchers and hobbyists working on Mamba-based manipulation policies, and it adds the first state space model policy to the LeRobot catalog so that any compatible dataset, robot, or hosted checkpoint can be used to drive it.

The empirical takeaway is narrower but useful. In a low data, two camera regime on a real arm, MTIL's additional capacity over ACT is not free: the same expressive recurrent state that promises long-horizon memory also makes the policy more prone to memorizing a dataset, and our results put MTIL behind ACT under similar conditions. We also observed that fail restart evaluation roughly triples or quadruples raw success rates for both policies, which is a cheap and underused diagnostic for whether a poor headline number reflects bad perception, bad control, or simply a dataset that never showed the policy what recovery looks like.

Future work falls into two natural directions. The first is data: collecting episodes that include deliberate failure to recovery trajectories, and scaling to four cameras so that MTIL's capacity has more visual signal to fit against. The second is regularization: experimenting with encoder dropout, partial encoder fine tuning, and live decode plus image augmentation as alternatives to the cached pipeline we used. This would help see whether MTIL's overfitting can be brought under control without sacrificing the consumer hardware training budget that motivated this work.

References

[1] Y. Zhou, Y. Lin, F. Peng, J. Chen, K. Huang, H. Yang, and Z. Yin, "MTIL: Encoding Full History with Mamba for Temporal Imitation Learning," *IEEE Robotics and Automation Letters*, vol. 10, no. 11, pp. 11761–11767, 2025, doi: 10.1109/LRA.2025.3615520.

[2] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn, "Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware," in *Proc. Robotics: Science and Systems (RSS)*, Daegu, Republic of Korea, Jul. 2023, doi: 10.15607/RSS.2023.XIX.016.

[3] A. Gu and T. Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," arXiv preprint arXiv:2312.00752, 2023.

[4] T. Dao and A. Gu, "Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality," in *Proc. International Conference on Machine Learning (ICML)*, 2024.

[5] M. Oquab, T. Darcet, T. Moutakanni, et al., "DINOv2: Learning Robust Visual Features without Supervision," *Transactions on Machine Learning Research*, 2024. [Online]. Available: https://openreview.net/forum?id=a68SUt6zFt

[6] R. Cadene, S. Alibert, A. Soare, Q. Gallouedec, A. Zouitine, S. Palma, P. Kooijmans, M. Aractingi, M. Shukor, D. Aubakirova, M. Russi, F. Capuano, C. Pascal, J. Choghari, J. Moss, and T. Wolf, "LeRobot: State-of-the-Art Machine Learning for Real-World Robotics in Pytorch," GitHub repository, 2024. [Online]. Available: https://github.com/huggingface/lerobot